

8 详细设计

8.1 系统总体设计

8.1.0 系统总体架构图



8.1.1 功能描述

WeCraft，是一套面向微信小程序开发场景的智能化软件工程生成系统。系统以自然语言需求为输入，以可导入微信开发者工具的完整小程序工程为输出，通过多智能体协同、工程目录规划、上下文压缩、页面四件套生成、一致性校验与错误反馈重试等机制，形成从“需求描述”到“工程落地”的自动化闭环。

与传统代码生成工具不同，本系统并不是简单地让大模型一次性输出代码，而是将真实软件开发流程拆解为需求分析、架构设计、目录规划、上下文整理、代码生成、工程校验等多个阶段，并为每个阶段配置具有明确职责的智能体。系统利用 CrewAI 的 Agent、Task、Crew、Process 等核心概念，将多个专业角色组织为一个具备任务分工能力、上下文传递能力和工程协同能力的生成团队，从而提升生成结果的稳定性、可维护性和工程完整度。

系统的整体目标是降低微信小程序开发门槛，使用户只需输入较为自然、模糊的系统想法，系统即可通过智能需求分析与多阶段生成流程，将其转化为结构化需求、系统架构和可运行代码。对于非专业开发者而言，该系统能够辅助其快速获得小程序原型；对于有一定开发经验的用户而言，该系统能够显著减少重复性页面搭建、样式编写和工程初始化工作，提高开发效率。

8.1.2 性能描述

本系统在设计中重点关注生成稳定性、工程一致性、上下文利用效率和用户交互体验。系统采用分阶段、分页面、分组件的生成方式，避免一次性生成完整系统时出现上下文过长、输出截断、文件缺失、页面不匹配等问题。

在性能设计上，系统具备以下特点：

1. 支持质量档位控制。系统提供 fast、balanced、quality 三种质量档位，分别对应不同的依赖上下文数量、重试次数、页面数量上限和校验强度，使用户可以根据生成速度、Token 成本和结果质量进行取舍。
2. 支持页面级生成与页面级重试。系统按页面四件套生成代码，当某个页面校验失败时，仅针对该页面进行反馈重试，避免全系统重复生成。
3. 支持上下文压缩与动态注入。系统不会将所有需求、架构和已生成代码全部塞入模型，而是根据当前生成任务动态选择最相关的上下文，降低大模型注意力分散和上下文污染问题。
4. 支持中间产物留存。系统将需求分析结果、架构设计结果、文件清单、上下文摘要、页面生成顺序、校验报告、生成文件摘要等内容保存到本地，便于用户复盘、调试和继续优化。
5. 支持 Web UI 交互。系统通过本地 Web 界面实现需求对话、任务启动、日志清洗、进度提示和生成状态展示，改善命令行生成方式的使用门槛。

8.1.3 输入

系统输入主要包括以下几类：

1. 用户自然语言需求。例如“我想做一个校园二手交易小程序”“我想做一个运动打卡小程序”“我想做一个宠物领养平台”等。
2. 用户在多轮对话中的补充说明，包括功能模块、页面需求、用户角色、数据来源、权限规则、风格偏好、是否需要云开发、是否需要本地模拟数据等。
3. 系统运行配置，包括模型 API Key、模型名称、API Base、质量档位、输出目录、最大页数、重试次数等。
4. 生成过程中的中间产物，包括需求分析结果、架构设计结果、文件规划结果、代码生成上下文、已生成文件内容、校验反馈信息等。

8.1.4 输出

系统输出包括两类：工程输出和过程输出。

工程输出为完整微信小程序系统，默认输出至 `generated_mini_program/` 目录。该目录中包含：

- `app.js`
- `app.json`
- `app.wxss`
- `project.config.json`
- `sitemap.json`
- `pages/` 页面目录
- `components/` 组件目录
- `utils/` 工具模块目录
- 其他根据系统需要生成的模块文件

过程输出保存至 `.crew_state/latest_generation/` 目录，主要包括：

- `manifest.json`：本次生成的运行参数。
- `input_requirement.md`：用户输入或 Web UI 汇总后的需求。
- `requirement_analysis.md`：需求分析阶段输出。
- `architecture.md`：架构设计阶段输出。
- `file_plan_raw.json`：模型原始文件清单。
- `file_plan_schema_report.json`：文件清单校验与修复报告。
- `file_plan_validated.json`：最终确认的文件清单。
- `codegen_context_raw.json`：原始代码生成上下文。
- `codegen_context.json`：规范化后的代码生成上下文。
- `page_generation_order.json`：页面生成顺序。
- `validation_report.json`：一致性校验与重试记录。
- `generated_files_summary.json`：生成文件摘要。

这些输出不仅保证用户能够获得最终工程，也使系统具备可追溯、可调试、可迭代的工程特征。

8.1.5 程序逻辑

系统总体程序逻辑如下：



该流程使系统具备明显的软件工程化特征。每个阶段都承担独立责任，上一个阶段的输出会作为下一个阶段的输入，从而形成稳定的流水线式生成过程。

8.1.6 限制条件

系统当前主要面向微信小程序前端工程生成，适合生成中小型小程序原型或具有较明确页面结构的应用。对于高度复杂的后端业务、强安全要求的支付系统、大规模数据库事务系统、实时协同系统等场景，仍需要开发者进行二次开发和人工审核。

同时，由于系统依赖大语言模型生成代码，生成结果仍可能受到模型理解能力、上下文长度、输出稳定性和接口可用性的影响。因此系统通过质量档位、一致性校验、中间产物留存和局部重试机制降低风险，但不能完全替代开发者的最终测试与验收。

8.2 多智能体协同架构设计

8.2.1 功能描述

本系统的核心创新之一在于引入基于 CrewAI 的多智能体协同架构。系统不再使用单一大模型直接承担全部任务，而是将微信小程序生成过程拆解为多个专业化角色，并为每个角色配置独立的目标、背景、提示词和输出规范。

系统中的主要 Agent 包括：

Agent 名称	角色定位	核心职责
user_agent	资深微信小程序需求分析师	与用户交互，明确需求，输出结构化需求文档
organizer_agent	资深微信小程序架构师	将需求转化为架构方案、页面设计和模块划分
file_planner_agent	微信小程序工程目录规划师	根据架构设计输出完整且尽量精简的工程文件清单
codegen_context_agent	代码生成上下文压缩师	将需求和架构压缩为面向代码生成的核心上下文
code_agent	微信小程序开发工程师	根据文件清单和上下文生成完整文件内容
validation 模块	工程一致性校验器	检查路径、事件、组件、跳转等工程一致性问题

这种设计方式模拟了真实软件开发团队的分工模式。需求分析师负责理解用户，架构师负责设计系统，目录规划师负责组织工程结构，上下文压缩师负责控制信息密度，开发工程师负责生成代码，校验器负责质量把关。各智能体之间职责清晰，避免单一模型同时承担过多任务导致输出混乱。

8.2.2 性能描述

多智能体架构的性能优势主要体现在任务分解和上下文隔离。单模型直接生成完整工程时，模型需要同时理解需求、设计架构、规划目录、编写代码和检查错误，任务负载过重，容易导致遗漏和幻觉。多智能体架构将复杂目标分解为多个较小任务，使每次模型调用都具有更明确的输入、目标和输出格式。

该架构具有以下优势：

- 降低单次推理复杂度。每个 Agent 只处理自己的专业任务，减少任务目标冲突。
- 提高中间结果可控性。需求、架构、目录、上下文和代码均有独立产物，可单独检查。
- 增强工程一致性。后续阶段严格依赖前序阶段输出，减少自由发挥导致的结构漂移。
- 便于扩展。未来可以继续加入 UI 设计 Agent、测试用例 Agent、后端接口 Agent、部署 Agent 等角色。

8.2.3 输入

多智能体协同架构的输入包括用户需求、上游 Agent 输出、系统配置参数以及校验反馈信息。例如，架构设计 Agent 的输入是需求分析 Agent 的输出；文件规划 Agent 的输入是架构设计结果；代码生成 Agent 的输入包括代码生成上下文、文件清单、当前文件用途和依赖文件摘要。

8.2.4 输出

不同 Agent 产生不同类型的结构化输出：

- 需求分析 Agent 输出需求分析报告。
- 架构设计 Agent 输出架构设计文档。
- 文件规划 Agent 输出 JSON 格式文件清单。
- 上下文压缩 Agent 输出 JSON 格式代码生成上下文。
- 代码生成 Agent 输出 JSON 格式文件内容。
- 校验模块输出校验报告和重试反馈。

系统通过统一的 JSON 输出约束保证机器可解析性，使大模型输出能够被后续 Python 脚本自动处理，而不是停留在文本描述层面。

8.2.5 程序逻辑

多智能体协同过程如下：



在实现中，系统使用 CrewAI 的 Agent 定义智能体角色，使用 Task 描述具体任务，使用 Crew 组织多个任务执行。每个任务都具有明确的 description、expected_output 和 agent，从而使每一次模型调用都具有强约束。

8.2.6 限制条件

多智能体架构虽然提升了系统可控性，但也带来了更多模型调用次数和更高流程复杂度。因此系统通过质量档位和上下文压缩机制控制成本。对于非常简单的小程序需求，完整多 Agent 流程可能显得较重；对于复杂系统，多 Agent 流程则能够显著提升结果质量。

8.3 需求分析智能体设计

8.3.1 功能描述

需求分析智能体是系统的第一类核心 Agent，负责将用户模糊的自然语言想法转化为结构化需求文档。用户通常不会直接提供完整的软件需求规格说明，而是输入较为口语化的描述。例如：

1 | 我想做一个校园跑腿小程序，可以发布任务，也可以接单。

需求分析 Agent 会围绕小程序开发真正需要的信息进行追问和整理，包括：

- 系统目标
- 用户角色
- 功能模块
- 页面与跳转逻辑
- 数据字段建议
- 技术实现要点
- 待确认项

该模块的目标是将“用户想法”转化为“可被架构设计使用的需求文档”。

8.3.2 性能描述

需求分析智能体采用多轮对话策略，每轮聚焦 2 到 5 个关键问题，避免一次性抛出过多问题造成用户负担。系统的问题设计主要围绕会影响代码生成的内容展开，而不是泛泛讨论商业背景。

该模块重点提升以下能力：

1. 需求收敛能力。通过多轮追问逐步减少需求不确定性。
2. 结构化表达能力。将用户回答整理为固定格式，便于后续架构 Agent 使用。
3. 默认假设能力。在 Web UI 一次性生成模式下，如果用户没有继续补充，系统会基于常见小程序模式进行合理补全。
4. 面向实现的分析能力。问题聚焦页面、角色、字段、权限、数据来源和边界流程。

8.3.3 输入

需求分析智能体的输入为用户原始需求及多轮补充信息。输入可以来自命令行，也可以来自 Web UI 对话系统。

8.3.4 输出

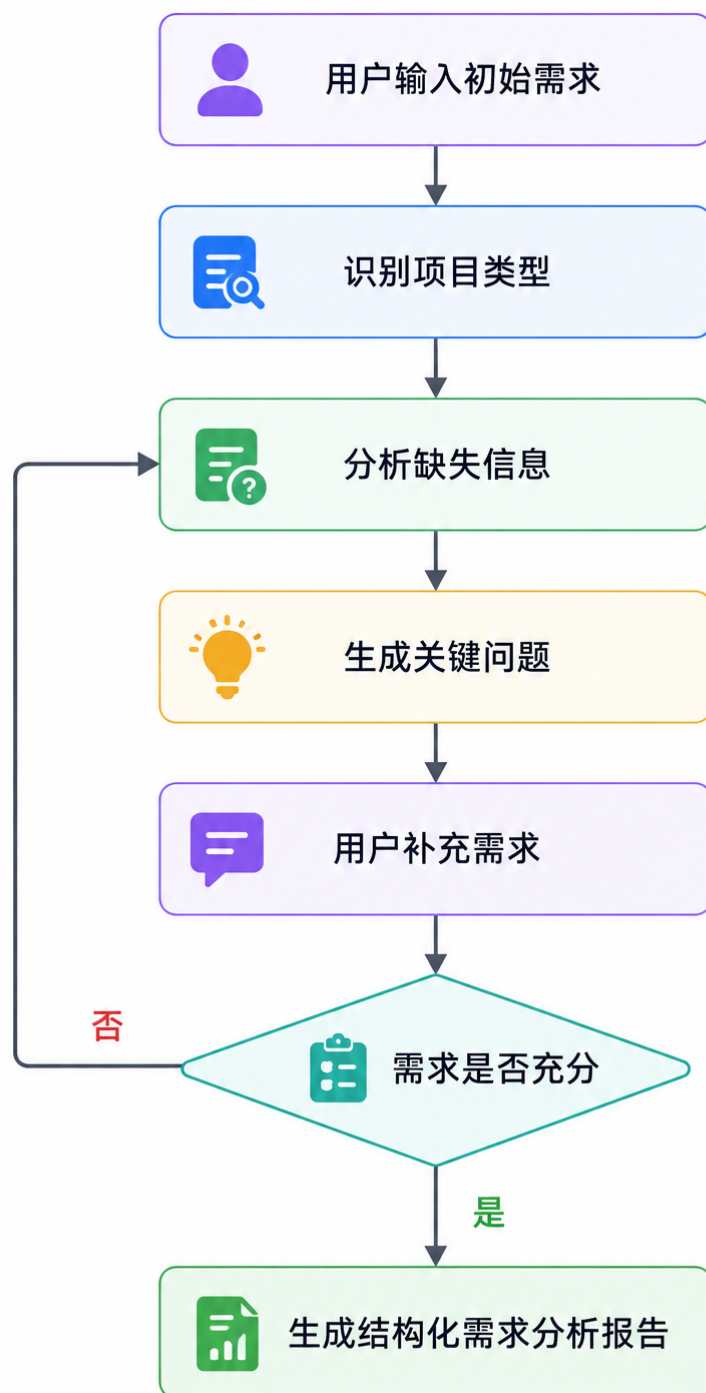
需求分析智能体输出结构化需求分析报告，主要包括：

- 1 | 系统目标
- 2 | 用户角色
- 3 | 功能模块
- 4 | 页面设计
- 5 | 交互逻辑
- 6 | 数据字段建议
- 7 | 技术实现要点
- 8 | 待确认项

该输出会保存为 `requirement_analysis.md`，并作为架构设计阶段的直接输入。

8.3.5 程序逻辑

需求分析流程如下：



在 Web UI 中，系统还提供专门的需求对话服务。该服务要求模型只返回 JSON，字段包括 `status`、`reply` 和 `plan`。当信息不足时，`status` 为 `ask`；当需求足够时，`status` 为 `plan`，系统即可进入正式生成流程。

8.3.6 限制条件

需求分析结果依赖用户输入质量。如果用户需求过于模糊且拒绝补充，系统只能根据常见模式进行默认假设。默认假设可能不完全符合真实业务场景，因此最终生成前仍建议用户检查需求方案。

8.4 架构设计智能体设计

8.4.1 功能描述

架构设计智能体负责将需求分析结果转化为可指导代码生成的系统架构方案。该 Agent 被定义为“资深微信小程序架构师”，其核心目标是完成页面结构设计、模块划分、数据结构设计、接口设计、用户流程设计和风险分析。

该模块输出内容包括：

- 系统架构设计
- 功能模块拆解
- 页面结构设计
- 数据结构设计
- API 设计
- 用户流程
- 技术选型说明
- 风险与优化建议

架构设计智能体相当于系统中的“技术负责人”，它决定后续代码生成的整体骨架。

8.4.2 性能描述

该模块重点关注架构合理性和页面数量控制。微信小程序系统如果页面拆分过细，会导致文件数量过多、跳转关系复杂、维护成本上升；如果页面合并过度，又会导致单页面逻辑复杂、职责不清。因此系统在架构 Agent 的提示词中加入页面合并与组件抽象策略。

设计规则包括：

1. 页面设计尽量精简，避免一个小功能拆一个页面。
2. 一个页面可以承载多个相近功能模块。
3. 新增和编辑优先合并为通用表单页。
4. 多类详情优先合并为通用详情页。
5. 多类列表优先合并为通用列表页。
6. 可复用 UI 优先抽象为组件，而不是新页面。
7. 中小型系统页面数量优先控制在 5 到 8 个。

这些规则使系统生成的小程序结构更加接近人工设计的工程结构。

8.4.3 输入

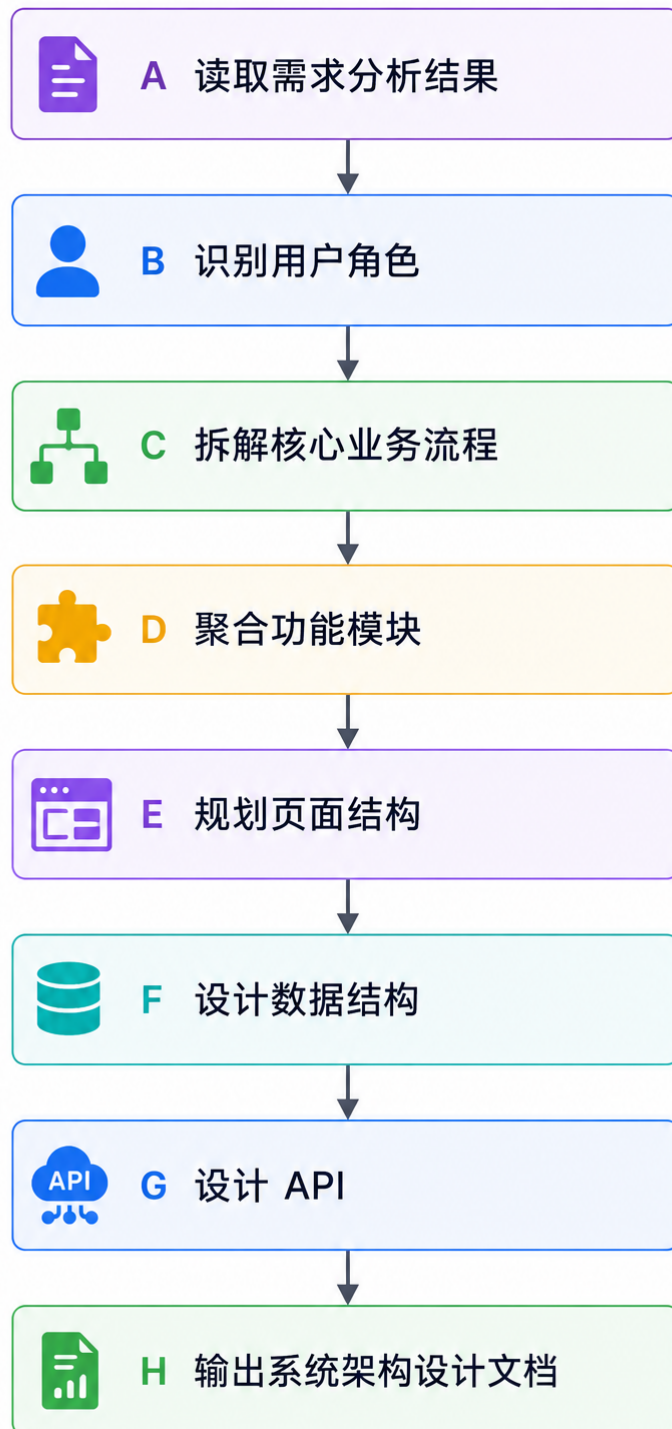
架构设计智能体的输入为需求分析 Agent 生成的结构化需求文档。

8.4.4 输出

架构设计智能体输出完整架构设计文档，并保存为 `architecture.md`。该文档会被文件规划 Agent 和上下文压缩 Agent 共同使用。

8.4.5 程序逻辑

架构设计流程如下：



8.4.6 限制条件

架构设计智能体主要面向前端小程序架构，对复杂后端微服务、数据库分布式事务、企业级权限系统等设计不会展开到生产级细节。如果系统需要对接真实后端或第三方服务，仍需开发者根据接口文档进行二次开发。

8.5 工程目录规划模块设计

8.5.1 功能描述

工程目录规划模块负责将架构设计文档转化为微信小程序工程文件清单。该模块由 `file_planner_agent` 完成，其角色为“微信小程序工程目录规划师”。

该模块的主要目标是生成尽可能精简但完整可运行的文件路径清单。系统要求文件清单必须包含：

- `app.js`
- `app.json`
- `app.wxss`
- 页面 `.js` 文件
- 页面 `.wxml` 文件
- 页面 `.wxss` 文件
- 页面 `.json` 文件
- 必要组件文件
- 必要工具模块文件

文件规划阶段的意义在于先确定工程结构，再进入代码生成。这样可以避免大模型边写代码边临时创造文件路径，导致后续页面引用混乱。

8.5.2 性能描述

该模块采用严格 JSON 输出格式，系统可直接解析和校验。每个文件对象必须包含：

- `path`
- `type`
- `purpose`

示例：

```
1 {
2   "files": [
3     {
4       "path": "pages/index/index.js",
5       "type": "page",
6       "purpose": "首页逻辑文件"
7     }
8   ]
9 }
```

系统还会对文件清单执行 Schema 校验、轻度修复、必要补齐和排序，确保后续生成阶段具有可靠输入。

8.5.3 输入

工程目录规划模块的输入为架构设计 Agent 输出的架构设计文档。

8.5.4 输出

输出为 JSON 格式系统文件清单，并生成以下中间文件：

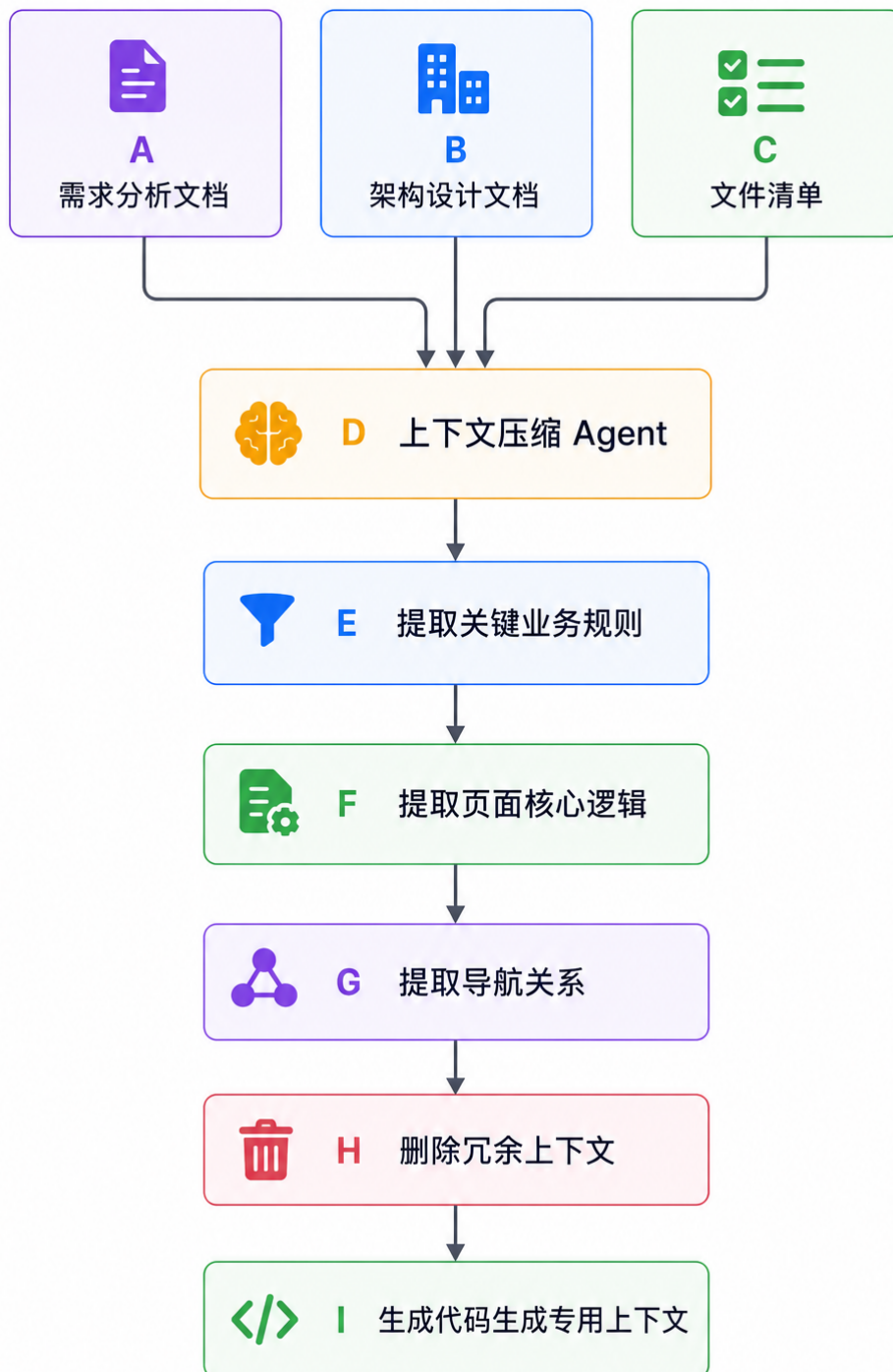
- `file_plan_raw.json`

- `file_plan_schema_report.json`
- `file_plan_validated.json`

其中 `file_plan_raw.json` 保存模型原始输出, `file_plan_schema_report.json` 保存校验与修复过程, `file_plan_validated.json` 保存最终确认后的文件清单。

8.5.5 程序逻辑

工程目录规划流程如下：



8.5.6 限制条件

文件规划依赖架构设计的准确性。如果架构设计阶段遗漏某些页面或模块，文件规划阶段可能无法完全补齐业务语义。因此系统在后续代码生成和校验阶段继续进行一致性检测，以降低前序遗漏带来的影响。

8.6 上下文压缩与整理模块设计

8.6.1 功能描述

上下文压缩与整理模块是本系统的重要创新点之一。该模块由 `codegen_context_agent` 完成，其角色为“代码生成上下文压缩师”。

由于大语言模型存在上下文长度限制和注意力衰减问题，如果将完整需求文档、完整架构文档、完整文件清单和所有已生成代码全部注入模型，不仅会造成 Token 消耗过高，还会使模型难以聚焦当前任务，增加生成错误。因此系统设计了专门的上下文压缩 Agent，将长文本资料整理为结构化、低冗余、高信息密度的代码生成上下文。

该模块输出内容包括：

- 系统摘要
- 页面摘要
- 数据模型摘要
- 导航关系摘要
- 工程约束摘要

8.6.2 性能描述

上下文压缩模块不是简单截断文本，而是基于代码生成需要进行语义压缩。系统要求该 Agent 删除冗余背景和泛化描述，但必须保留会影响代码质量的业务规则、字段、状态、页面交互和边界流程。

相比普通文本截断，该设计具有以下优势：

1. 保留关键业务信息。
2. 删除重复叙述。
3. 形成稳定 JSON 结构。
4. 便于后续按页面动态检索。
5. 降低上下文污染。
6. 减少 Token 成本。

系统支持不同质量档位下的上下文长度控制。例如 fast 档位注重速度和成本，quality 档位保留更多上下文并执行更强校验。

8.6.3 输入

上下文压缩模块输入包括：

- 需求分析结果
- 架构设计结果
- 系统质量档位配置

8.6.4 输出

该模块输出 JSON 格式代码生成上下文，示例结构如下：

```
1 {
2   "project_summary": "一句话系统概述",
3   "pages": [
4     {
5       "path": "pages/index/index",
```



```

6      "purpose": "页面用途",
7      "core_logic": "页面核心逻辑",
8      "ui_notes": "页面结构或 UI 重点"
9    }
10 ],
11 "data_model": "关键数据结构摘要",
12 "navigation": "页面跳转关系摘要",
13 "constraints": "代码实现必须遵守的重要约束"
14 }

```

输出会保存为：

- `codegen_context_raw.json`
- `codegen_context.json`

8.6.5 程序逻辑

上下文压缩流程如下：

```

1  读取需求分析结果和架构设计结果
2      ↓
3  提取系统目标和核心业务规则
4      ↓
5  提取页面用途和页面核心逻辑
6      ↓
7  提取关键数据结构
8      ↓
9  提取页面跳转关系
10     ↓
11  提取工程约束
12     ↓
13  删除冗余背景信息
14     ↓
15  输出代码生成专用上下文 JSON

```

8.6.6 限制条件

上下文压缩需要在“压缩率”和“信息完整性”之间平衡。如果压缩过度，代码生成可能缺少细节；如果保留过多，则会增加 Token 成本并降低模型聚焦能力。因此系统通过质量档位调节上下文保留程度。

8.7 页面四件套生成模块设计

8.7.1 功能描述

页面四件套生成模块负责生成微信小程序页面代码。微信小程序页面通常由四类文件组成：

- `.js`：页面逻辑
- `.wxml`：页面结构
- `.wxss`：页面样式
- `.json`：页面配置

传统逐文件生成方式容易出现页面内部不一致，例如 WXML 中绑定了 `bindtap="goDetail"`，但 JS 中没有 `goDetail` 方法；WXSS 中没有对应样式类；JSON 中组件引用路径错误。为解决该问题，本系统提出页面四件套成组生成机制。

系统在生成页面时，不是分别生成四个文件，而是将同一页面的 `.js`、`.wxml`、`.wxss`、`.json` 作为一个整体任务提交给代码生成 Agent，使模型在同一次推理中统一考虑页面状态、事件、结构、样式和配置。

8.7.2 性能描述

页面四件套生成机制显著提升页面内部一致性。系统要求代码生成 Agent 必须一次性输出指定页面目录下的全部文件，并且 `files` 列表只能包含任务要求的路径，不允许额外创建未规划文件。

该模块具有以下特点：

1. 页面内事件统一。WXML 中绑定的事件必须在 JS 中实现。
2. 页面内数据统一。WXML 中使用的数据字段必须在 JS 的 `data` 中定义或通过逻辑赋值。
3. 页面内样式统一。WXML 中使用的主要 class 应在 WXSS 中有对应样式。
4. 页面配置统一。页面 JSON 中的标题、组件引用应与页面功能一致。
5. 路由关系统一。页面跳转路径必须来自文件清单和 `app.json` 页面配置。

8.7.3 输入

页面四件套生成模块的输入包括：

- 当前页面目录
- 当前页面四件套文件路径
- 当前页面用途说明
- 代码生成上下文
- 系统文件清单
- 直接依赖上下文
- 上一轮校验反馈

8.7.4 输出

输出为严格 JSON，格式如下：

```

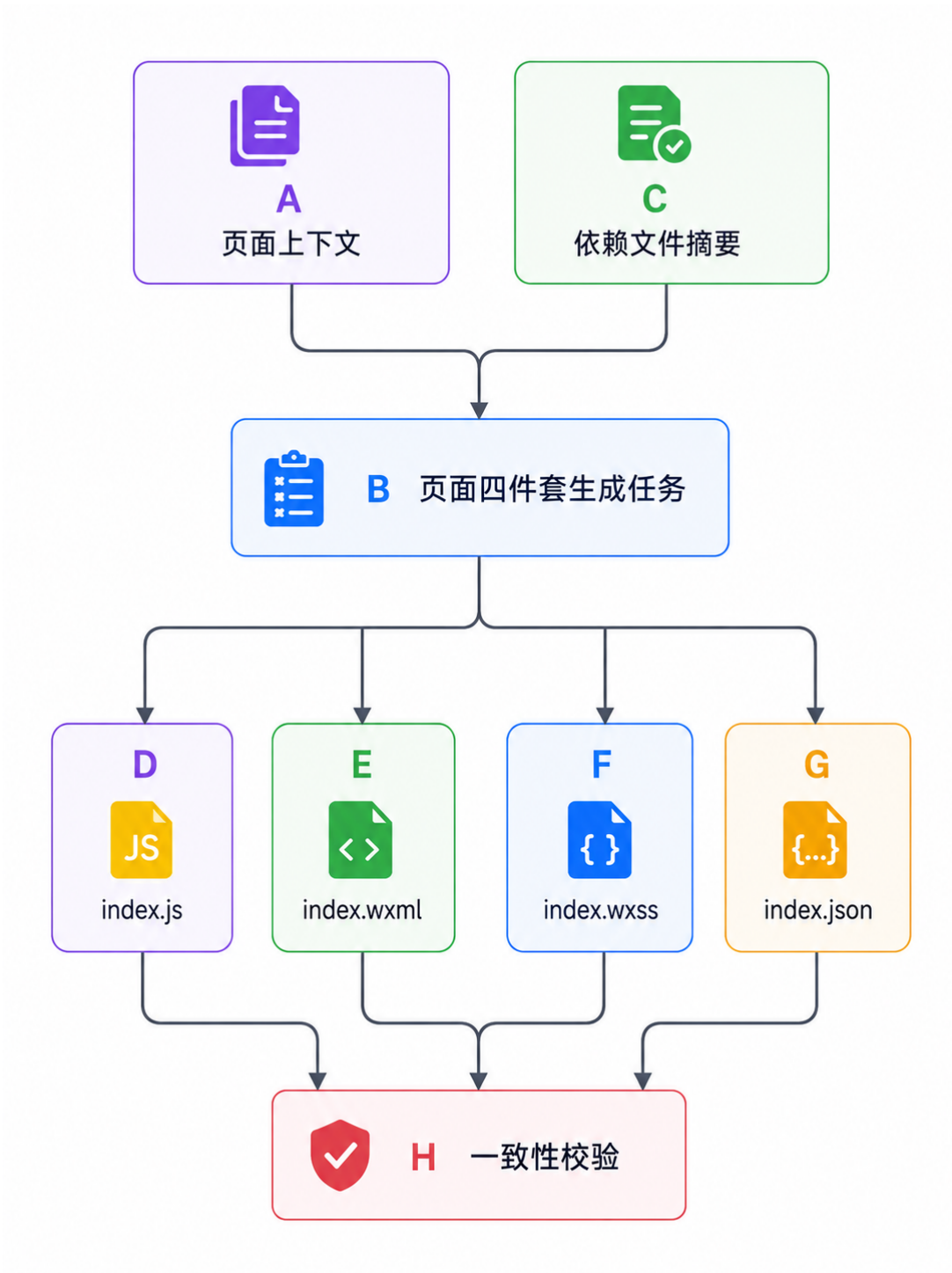
1 {
2   "files": [
3     {"path": "pages/index/index.js", "content": "完整 JS 代码"},
4     {"path": "pages/index/index.wxml", "content": "完整 WXML 代码"},
5     {"path": "pages/index/index.wxss", "content": "完整 WXSS 代码"},
6     {"path": "pages/index/index.json", "content": "完整 JSON 配置"}
7   ]
8 }

```

系统解析该 JSON 后，将文件写入输出目录。

8.7.5 程序逻辑

页面生成流程如下：



8.7.6 限制条件

页面四件套生成能够提升页面内部一致性，但仍不能完全保证业务逻辑完全正确。对于复杂算法、真实网络请求、支付流程、地图定位、文件上传等特殊能力，仍需要开发者结合微信官方文档进行测试与完善。

8.8 组件与支持文件生成模块设计

8.8.1 功能描述

除页面代码外，系统还支持生成组件、工具模块和页面私有模块。该模块用于降低页面代码复杂度，提高工程复用性。

系统会优先生成：

- `components/` 下的公共组件
- `utils/` 下的工具函数
- `pages/**/modules` 下的页面私有逻辑模块

这些支持文件会先于页面生成，为页面代码提供稳定依赖。

8.8.2 性能描述

支持文件预生成机制可以减少页面生成阶段的接口不一致问题。如果页面先生成，而组件和工具函数后生成，页面可能会臆造不存在的 API。系统通过预先生成依赖文件，使页面生成时能够引用已有文件摘要，从而提高依赖一致性。

该模块具有以下优势：

1. 降低页面文件体积。
2. 提高公共逻辑复用率。
3. 减少重复代码。
4. 降低页面与工具模块 API 不一致概率。
5. 便于后续人工维护。

8.8.3 输入

输入包括文件清单、文件用途说明、代码生成上下文和已有依赖文件摘要。

8.8.4 输出

输出为单文件 JSON：

```
1 {  
2   "path": "utils/storage.js",  
3   "content": "完整文件内容"  
4 }
```

8.8.5 程序逻辑

支持文件生成流程如下：

```
1  读取文件清单
2      ↓
3  筛选 components / utils / modules 文件
4      ↓
5  根据依赖顺序优先生成支持文件
6      ↓
7  保存文件内容
8      ↓
9  供页面生成阶段引用
```

8.8.6 限制条件

支持文件生成依赖文件清单的准确性。如果文件规划阶段未将公共逻辑抽象为组件或工具函数，系统可能会将更多逻辑写入页面文件中，导致页面代码较长。

8.9 一致性校验与自动重试模块设计

8.9.1 功能描述

一致性校验模块是系统保证生成质量的重要环节。该模块通过规则检查和工程结构分析，对模型生成的代码进行自动检测，发现问题后将错误信息反馈给代码生成 Agent 并触发局部重试。

系统重点检查以下问题：

- 页面路径是否与 `app.json` 一致。
- 页面跳转路径是否存在。
- WXML 绑定事件是否在 JS 中实现。
- 页面是否错误使用 `wx.switchTab`。
- 页面 JSON 中组件引用是否存在。
- 是否出现文件清单外路径。
- 是否调用被限制的云开发能力。
- 数据字段、事件命名、页面引用是否明显不一致。

该模块使系统具备初步的“自我诊断”和“自我修复”能力。

8.9.2 性能描述

一致性校验模块采用分级策略。不同质量档位对应不同校验强度：

质量档位	校验强度	特点
fast	basic	快速检查关键错误
balanced	balanced	平衡速度与质量
quality	strict	更严格检查一致性问题

系统支持多轮重试。若当前页面生成结果未通过校验，系统会生成清晰的错误反馈，例如：

- 1
- 路径一致性错误：当前 `app.json` 模板未配置 `tabBar`，页面 JS 不应调用 `wx.switchTab`；请改用 `navigateTo`、`redirectTo` 或 `reLaunch`。

随后系统将该反馈注入下一轮页面生成任务，使模型针对性修复。

8.9.3 输入

一致性校验模块输入包括：

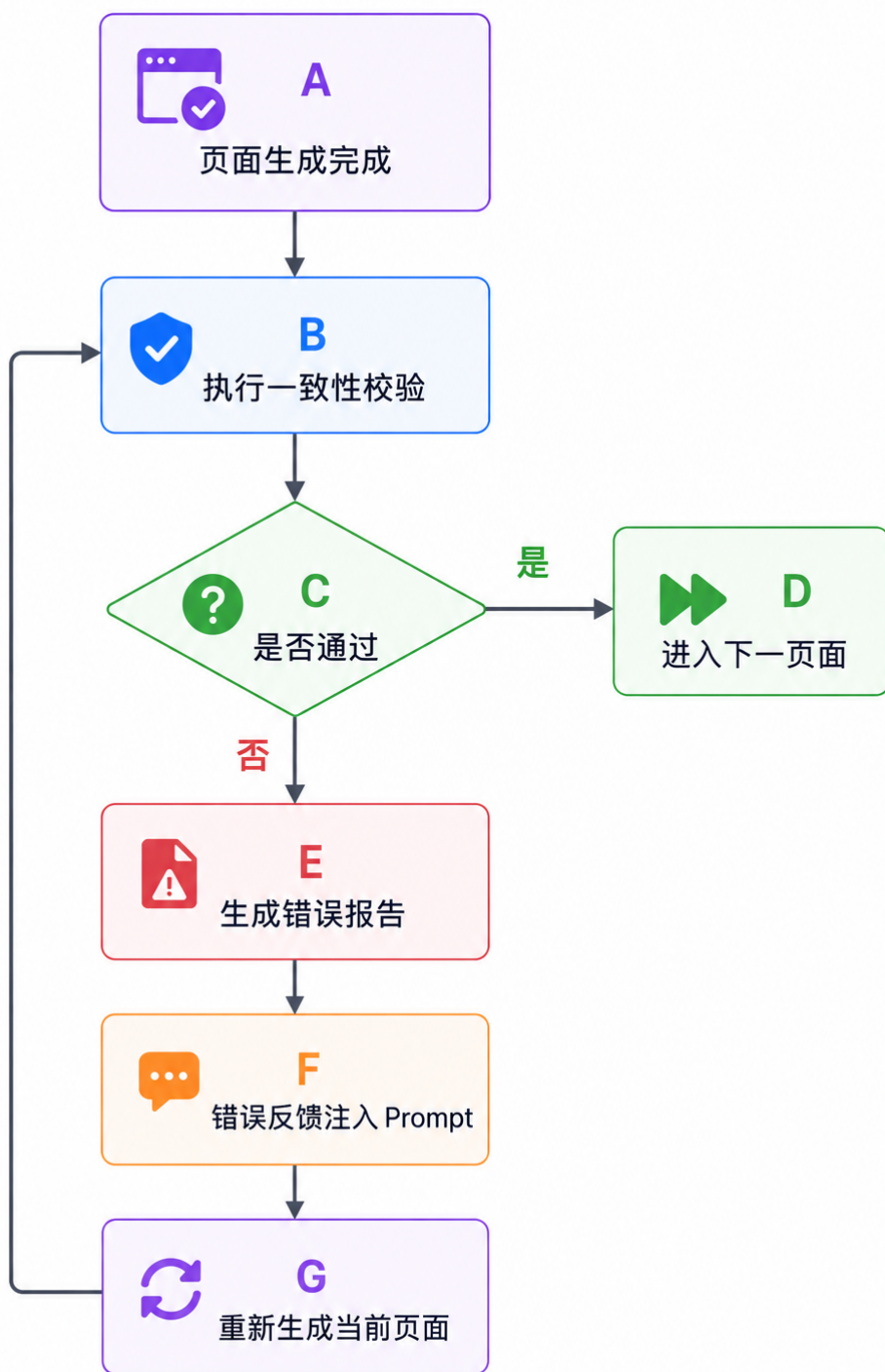
- 当前页面四件套内容
- `app.json` 页面配置
- 最终文件清单
- 已生成文件集合
- 当前质量档位

8.9.4 输出

输出为校验结果和错误反馈。若通过校验，则页面进入下一阶段；若未通过，则生成错误报告并触发重试。所有校验过程会记录到 `validation_report.json`。

8.9.5 程序逻辑

一致性校验流程如下：



8.9.6 限制条件

当前一致性校验主要基于静态规则和工程结构检查，无法完全替代运行时测试。例如复杂条件渲染、异步数据加载、用户权限状态、网络异常等问题仍需在微信开发者工具中进行实际运行测试。

8.10 固定模板生成模块设计

8.10.1 功能描述

为保证基础工程稳定性，系统对部分全局文件采用固定模板生成，而不是完全交由大模型自由生成。固定模板主要包括：

- `app.js`
- `app.json`
- `app.wxss`
- `project.config.json`
- `sitemap.json`

其中 `app.json` 会根据最终页面清单自动配置页面路径，保证小程序入口结构与文件清单一致。

8.10.2 性能描述

固定模板机制能够降低基础工程错误概率。大模型在生成全局配置文件时，容易出现 JSON 格式错误、页面路径遗漏、无效字段、实验性配置不兼容等问题。系统通过模板生成关键配置文件，可以保证系统至少具备稳定的工程入口。

该机制具有以下优势：

1. 保证全局配置格式正确。
2. 保证页面路径与文件清单同步。
3. 减少模型在基础配置上的不确定性。
4. 降低微信开发者工具导入失败概率。
5. 便于后续根据平台要求统一调整配置。

8.10.3 输入

固定模板模块输入包括：

- 最终文件清单
- 页面目录集合
- 系统名称
- 运行环境配置

8.10.4 输出

输出为基础小程序配置文件和模板文件。

8.10.5 程序逻辑

固定模板生成流程如下：

```
1  读取最终文件清单
2      ↓
3  提取 pages 页面路径
4      ↓
5  生成 app.json
6      ↓
7  生成 app.js
8      ↓
9  生成 app.wxss
10     ↓
11 生成 project.config.json
12     ↓
13 生成 sitemap.json
```

8.10.6 限制条件

固定模板更适合通用小程序工程。若用户需要特殊配置，例如 tabBar、分包、skyline 渲染、插件配置、权限声明、地理位置接口等，则需要系统根据需求进一步扩展模板生成逻辑。

8.11 Web UI 交互模块设计

8.11.1 功能描述

系统提供本地 Web UI，提升用户使用体验。相比命令行模式，Web UI 更适合普通用户进行需求表达、方案确认和生成过程观察。

Web UI 主要功能包括：

- 对话式需求分析
- 需求方案整理
- 一键启动生成流程
- 生成质量档位选择
- 实时日志显示
- 生成进度提示
- 输出目录展示
- 乱码和控制字符清洗

该模块使系统从“开发者脚本”升级为“可交互产品原型”。

8.11.2 性能描述

Web UI 使用本地 HTTP 服务实现，默认运行于 `127.0.0.1:7860`。系统采用任务状态共享结构记录当前运行状态，包括：

- 是否正在运行
- 当前状态
- 当前提示消息
- 日志内容
- 输出目录
- 质量档位
- 对话记录
- 需求草稿
- 生成方案
- 是否可以开始生成

系统通过前端轮询方式获取状态变化，并将后台日志转化为用户可读的进度消息。例如当日志中出现“开始需求分析”“架构设计完成”“正在生成页面四件套”等信息时，UI 会自动追加进度提示。

8.11.3 输入

Web UI 输入包括：

- 用户对话文本
- 是否强制生成方案
- 质量档位选择
- 生成启动指令

8.11.4 输出

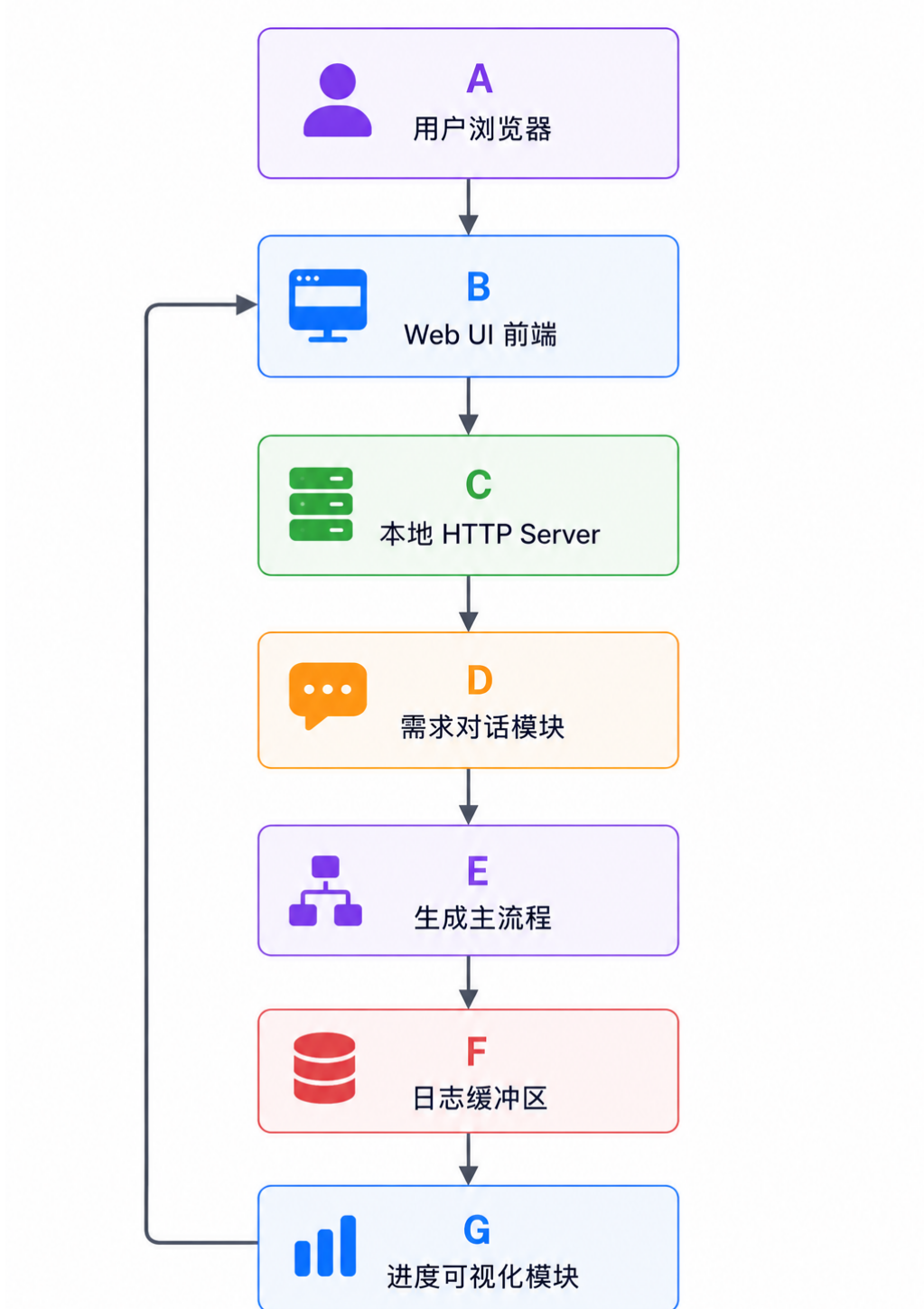
Web UI 输出包括：

- AI 需求追问
- 完整生成方案

- 实时进度消息
- 后台运行日志
- 输出目录位置
- 错误提示信息

8.11.5 程序逻辑

Web UI 交互流程如下：



8.11.6 限制条件

当前 Web UI 主要面向本地单用户使用，不适合直接作为公网多人服务部署。如果未来要扩展为在线平台，需要进一步加入用户系统、任务队列、文件隔离、权限控制、API 限流和安全审计等机制。

8.12 日志清洗与进度可视化模块设计

8.12.1 功能描述

在 CrewAI 和大模型调用过程中，控制台会产生大量运行日志，其中可能包含 ANSI 控制字符、乱码字符、调试信息和较长的中间文本。为了提高用户体验，系统在 UI 层设计了日志清洗与进度可视化模块。

该模块负责：

- 清理 ANSI 转义字符
- 清理控制字符
- 修复部分乱码
- 截断过长日志
- 从日志中提取关键进度
- 将技术日志转化为用户可理解的状态消息

8.12.2 性能描述

系统设置最大日志长度，避免日志无限增长造成内存占用过高。日志清洗模块会对每次输出进行处理，然后写入共享状态。

系统内置多个进度识别规则，例如：

- 识别质量档位
- 识别需求分析开始
- 识别需求分析完成
- 识别架构设计开始
- 识别架构设计完成
- 识别文件清单生成
- 识别上下文压缩完成
- 识别页面生成
- 识别一致性重试
- 识别生成完成

8.12.3 输入

输入为后台生成流程输出到 stdout 或 stderr 的日志文本。

8.12.4 输出

输出为清洗后的日志和用户可读进度消息。

8.12.5 程序逻辑

日志处理流程如下：



8.12.6 限制条件

日志进度识别依赖固定文本规则。如果后续生成流程中的日志格式发生较大变化，需要同步更新进度识别规则。

8.13 中间产物管理模块设计

8.13.1 功能描述

中间产物管理模块负责保存生成过程中的关键数据，使系统具备可追溯性和可调试性。传统 AI 代码生成工具通常只给出最终代码，用户难以判断问题到底发生在需求理解、架构设计、文件规划还是代码生成阶段。本系统通过中间产物留存解决该问题。

8.13.2 性能描述

每次生成时，系统会将关键过程数据写入 `.crew_state/latest_generation/` 目录。用户可以通过查看这些文件分析生成质量：

- 如果需求分析结果偏离用户意图，说明问题发生在需求阶段。
- 如果页面结构不合理，说明问题发生在架构阶段。
- 如果文件路径缺失，说明问题发生在文件规划阶段。
- 如果页面代码错误，说明问题发生在代码生成或校验阶段。

这种设计使系统具备工程调试能力，而不是一次性黑盒生成。

8.13.3 输入

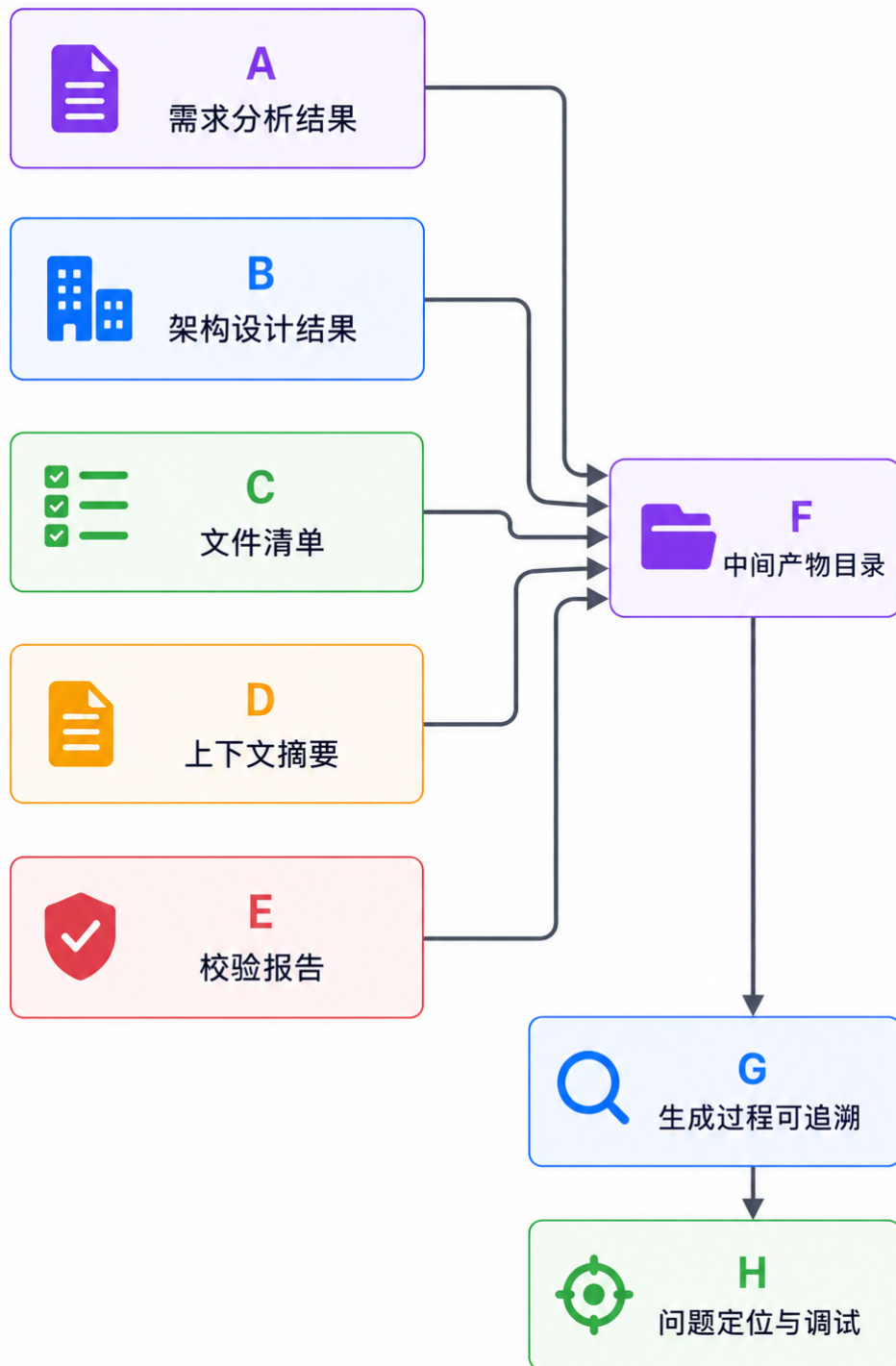
输入为各阶段 Agent 输出、系统配置和校验结果。

8.13.4 输出

输出为本地中间产物文件集合。

8.13.5 程序逻辑

中间产物保存流程如下：



8.13.6 限制条件

中间产物可能包含用户输入的需求描述和生成代码，因此如果系统未来部署为多人在线系统，需要对中间产物进行用户隔离、权限控制和敏感信息清理。

8.14 质量档位控制模块设计

8.14.1 功能描述

质量档位控制模块用于在生成速度、Token 成本和生成质量之间进行平衡。系统提供三种预设档位：

- fast
- balanced
- quality

不同档位会影响：

- 最大依赖上下文数量
- 页面数量上限
- 重试次数
- 上下文长度限制
- 校验强度

8.14.2 性能描述

三种档位适用场景如下：

档位	特点	适用场景
fast	上下文少，速度快，成本低	简单原型、快速测试
balanced	速度与质量均衡	普通小程序生成
quality	上下文多，重试多，校验严格	较复杂系统、参赛展示系统

该模块使系统具有可配置性，不同用户可以根据自身需求选择合适策略。

8.14.3 输入

输入为环境变量 `GENERATION_QUALITY` 或 Web UI 中选择的质量档位。

8.14.4 输出

输出为系统运行配置，包括：

- `MAX_DEP_CONTEXT`
- `RETRY_TIMES`
- `MAX_PAGE_COUNT`
- `CONTEXT_LIMITS`
- `VALIDATION_LEVEL`

8.14.5 程序逻辑

质量档位配置流程如下：

```
1  读取用户选择或环境变量
2      ↓
3  判断是否属于 fast / balanced / quality
4      ↓
5  若无效则回退 balanced
6      ↓
7  加载对应配置
8      ↓
9  影响后续上下文、重试和校验策略
```

8.14.6 限制条件

质量档位不能完全决定最终生成质量。模型能力、用户需求复杂度、提示词质量和 API 稳定性仍会影响最终结果。

8.15 模型接口与 JSON 输出控制设计

8.15.1 功能描述

系统使用大语言模型作为核心生成能力，并通过 CrewAI 的 `LLM` 配置接入 DeepSeek 模型。系统区分普通文本模型和 JSON 输出模型：

- 普通 LLM 用于需求分析和架构设计等自然语言输出任务。
- JSON LLM 用于文件规划、上下文压缩和代码生成等需要机器解析的任务。

8.15.2 性能描述

系统在 JSON 任务中启用 `response_format={"type":"json_object"}`，并在 Prompt 中反复强调“只能输出 JSON，不允许解释性文字，不允许 Markdown”。这样可以降低解析失败概率，使模型输出能够被程序自动处理。

同时系统设置了：

- temperature
- max_tokens
- timeout
- api_base
- api_key

等参数，以控制输出稳定性、输出长度和调用超时。

8.15.3 输入

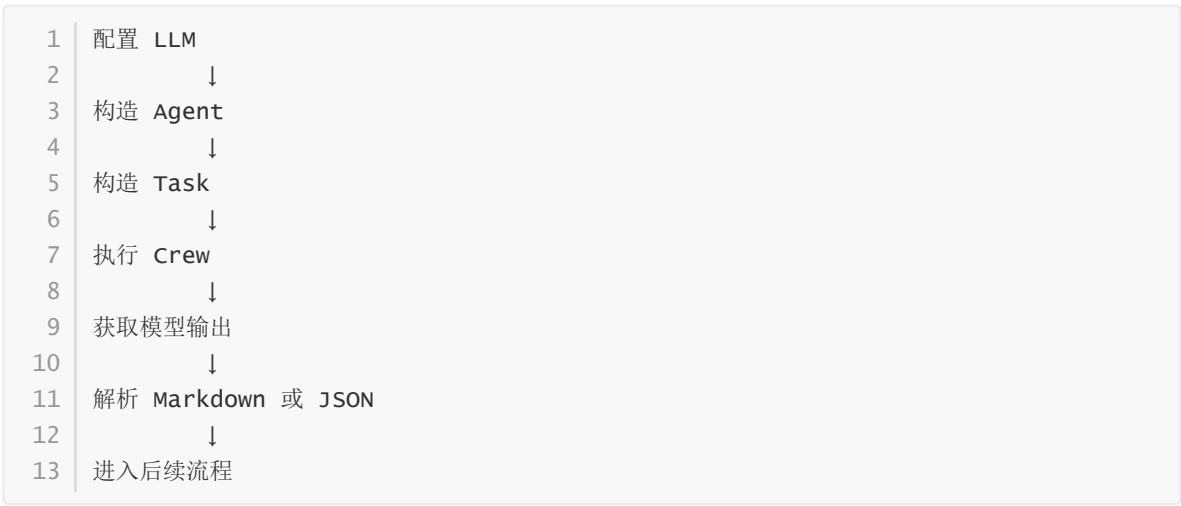
输入包括用户需求、任务描述、系统提示词、上下文和模型配置。

8.15.4 输出

输出根据任务不同分为 Markdown 文档、JSON 文件清单、JSON 上下文和 JSON 代码内容。

8.15.5 程序逻辑

模型调用逻辑如下：



8.15.6 限制条件

模型输出仍可能出现格式错误、字段缺失或内容不完整。因此系统在解析阶段加入 JSON 修复、Schema 校验和错误处理机制，以提高鲁棒性。

8.16 安全性与工程约束设计

8.16.1 功能描述

系统在代码生成过程中设置多项工程约束，防止模型生成不符合预期的小程序代码。例如在某些配置下，系统会限制生成云开发相关调用，避免用户没有配置云环境时出现运行错误。

系统重点约束包括：

- 页面路径必须与 app.json 一致。
- 页面跳转方式必须符合页面类型。
- 未配置 tabBar 时不得使用 `wx.switchTab`。
- 不得创建文件清单外路径。
- 页面四件套必须相互匹配。
- 组件引用必须存在。
- JSON 文件必须格式正确。
- 不得输出省略代码或伪代码。

8.16.2 性能描述

这些约束通过 Prompt、代码解析和校验规则共同实现。Prompt 负责前置约束模型行为，校验规则负责后置检查实际结果。

前置约束和后置校验结合，可以减少以下问题：

- 模型自由发挥创建不存在的页面。
- WXML 调用了 JS 中不存在的事件。
- 页面使用错误跳转 API。
- 生成代码被省略或截断。
- JSON 配置无法解析。

8.16.3 输入

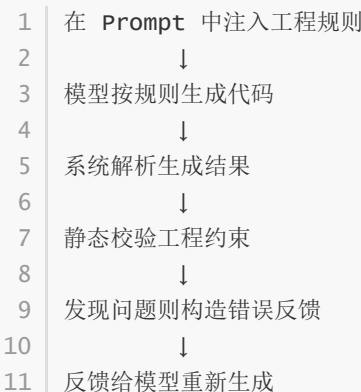
输入为系统工程约束、文件清单、生成代码和校验配置。

8.16.4 输出

输出为通过约束的代码文件，或包含错误原因的校验反馈。

8.16.5 程序逻辑

工程约束应用流程如下：



8.16.6 限制条件

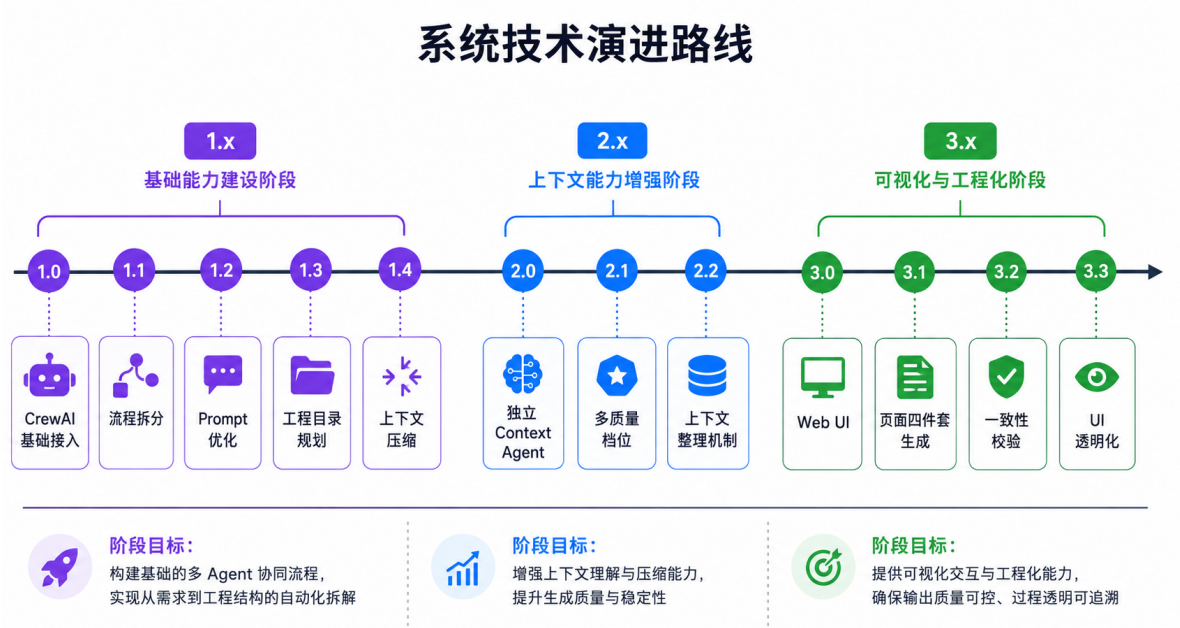
工程约束主要覆盖常见静态错误，无法完全覆盖所有运行时异常。最终仍需要用户在微信开发者工具中进行预览、调试和真机测试。

8.17 系统迭代过程设计

8.17.1 功能描述

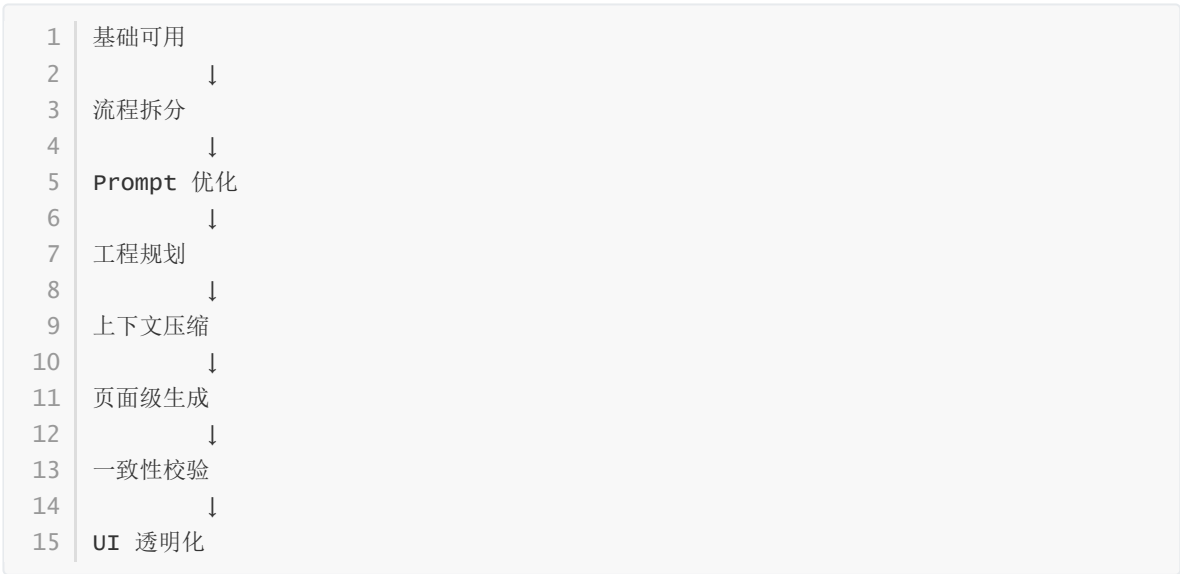
本系统经历了从简单线性 Agent 到多阶段工程化生成系统的迭代过程。迭代过程体现了系统从“能生成”到“生成得稳定、可控、可追溯”的技术演进。

8.17.2 迭代阶段



8.17.3 程序逻辑

系统迭代逻辑可以概括为：



8.17.4 创新价值

该迭代过程体现出系统不是简单堆叠 API，而是在持续解决 AI 工程生成中的真实问题，包括上下文限制、文件一致性、页面跳转错误、生成结果不可追溯和用户交互体验不足等。

8.18 创新性设计总结

8.18.1 多智能体协同创新

本系统将 CrewAI 多智能体框架引入微信小程序自动生成领域，将真实软件开发团队中的需求分析师、架构师、工程规划师、开发工程师等角色映射为多个 Agent，实现了面向软件工程流程的智能体协同生成。相比单模型代码生成，该方式具有更强的结构化能力和工程可控性。

8.18.2 软件 engineered 流程创新

系统不是直接生成代码，而是形成：

1 | 需求 → 架构 → 文件清单 → 上下文 → 代码 → 校验 → 重试

的完整工程流水线。该流程使 AI 生成从“文本生成”提升为“工程生成”，体现了 AI 软件工程自动化的思想。

8.18.3 上下文管理创新

系统提出面向代码生成的上下文压缩与动态注入机制，解决了长上下文条件下 Token 消耗高、模型注意力分散和代码不一致的问题。该机制使系统能够在有限上下文中保留最关键的逻辑和工程约束。

8.18.4 页面四件套生成创新

系统针对微信小程序特性提出页面四件套成组生成机制，将 `.js`、`.wxml`、`.wxss`、`.json` 作为统一任务生成，从源头降低页面内部事件、样式、数据和配置不一致问题。

8.18.5 一致性校验创新

系统通过静态规则和工程分析，对生成代码进行自动校验，并将错误反馈给模型进行局部重试，使系统具备初步的 AI 自修复能力。

8.18.6 透明化交互创新

系统提供 Web UI，对需求分析、架构设计、文件规划、页面生成和错误重试过程进行可视化展示，降低用户使用门槛，增强生成过程透明度。

8.19 实用性设计总结

8.19.1 面向普通用户的低门槛开发

用户无需掌握完整微信小程序工程结构，只需要描述系统想法，系统即可逐步引导其补充需求，并生成可运行工程。这对于课程设计、创新比赛、系统原型验证等场景具有较强实用价值。

8.19.2 面向开发者的效率提升

对于具备开发经验的用户，系统可以承担大量重复性工作，例如页面模板搭建、样式初始化、数据 mock、基础跳转逻辑、组件拆分等，使开发者能够将更多精力投入到业务创新和后续优化中。

8.19.3 面向教学场景的辅助价值

该系统可以帮助初学者理解小程序系统结构、页面四件套关系、路由配置、组件引用和前端工程组织方式，具有一定的教学辅助意义。

8.19.4 面向竞赛系统的原型构建价值

在创新比赛中，团队通常需要快速构建系统 Demo。该系统能够将想法快速转化为可展示的小程序原型，有助于提升系统表达能力和演示效果。

8.20 本章小结

本章从系统总体设计、多智能体协同架构、需求分析智能体、架构设计智能体、工程目录规划、上下文压缩、页面四件套生成、组件与支持文件生成、一致性校验、固定模板、Web UI、中间产物管理、质量档位控制、模型接口、安全约束和技术迭代等方面，对本系统的详细设计进行了说明。

本系统的核心并不是简单调用大模型生成代码，而是构建了一套基于 CrewAI 的多智能体软件工程生成系统。系统通过多角色分工、任务链编排、上下文压缩、页面级生成、规则校验和错误反馈重试，实现了从自然语言需求到微信小程序工程代码的自动化生成闭环。

从技术创新角度看，本系统融合了多智能体协同、Prompt Engineering、结构化 JSON 输出、上下文压缩、静态一致性校验和 Web UI 可视化等技术，体现了 AI Agent 在软件工程自动化方向的探索价值。

从实际应用角度看，本系统能够降低微信小程序开发门槛，提高原型开发效率，辅助学生团队、课程系统和创新竞赛团队快速完成小程序工程搭建，具有较强的实用性和扩展潜力。